Understanding Asynchronous Javascript

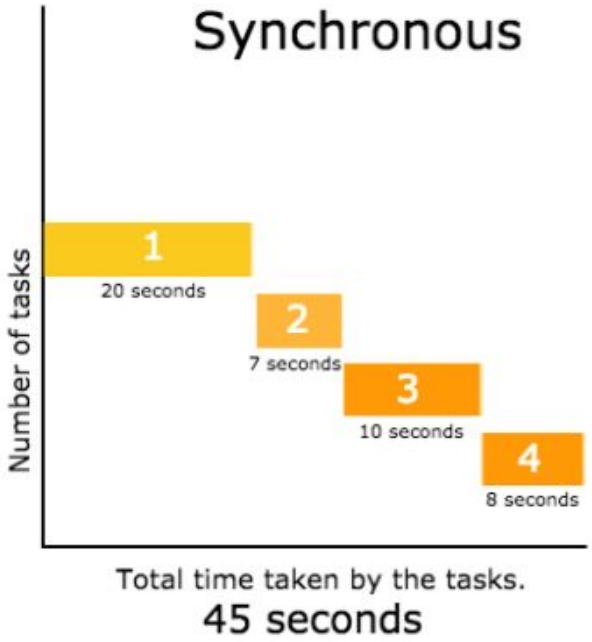
Yash Jain

SYNCHRONOUS JAVASCRIPT

- JavaScript is a synchronous, single-threaded language by default. This means code is executed line-by-line in a predictable sequence, where each task must finish before the next one begins
- Single threaded : javascript has only one call stack and can only do one task at time.
- Blocking nature : If a specific line of code takes a long time to execute , it "blocks" all subsequent code from running until it is finished .
- Analogy: Standing in a long single queue at bank or single-lane road where every car must wait for the one in front of it to move. If one car breaks down or stops to unload, the entire line stalls. Such blocking behavior can cause slow user experience . Similarly, long running synchronous tasks can make browser completely unresponsive.

ASYNCHRONOUS JAVASCRIPT

- Asynchronous javascript allows tasks to run in background without blocking the main execution thread.
- It allows concurrent operations, improved responsiveness, and efficient handling of time-consuming operations in web applications.
- Javascript uses three methods to handle asynchronous operations - Callbacks, Promises and Async/Await
- Analogy: Highway system that allows cars to pull into a bypass lane so traffic can keep flowing . Cooking multiple dishes simultaneously.


Synchronous
Number of tasks
1
20 seconds
2
7 seconds
3
10 seconds
4
8 seconds
Total time taken by the tasks.
45 seconds

Asynchronous
Number of tasks
1
20 seconds
2
7 seconds
3
10 seconds
4
8 seconds
Total time taken by the tasks.
20 seconds

Code:

Synchronous js :-

```
C: > Users > Yash Jain > koss > code1.js
console.log("first");
console.log("second");
console.log("third");
```

```
PS C:\Users\Yash Jain> cd koss
PS C:\Users\Yash Jain\koss> node code1.js
first
second
third
PS C:\Users\Yash Jain\koss>
```

Asynchronous js :-

```
C: > Users > Yash Jain > koss > code1.js > ...
console.log("first");
setTimeout(()=>{
    console.log("second");
},2000);
console.log("third");
```

```
PS C:\Users\Yash Jain\koss> node code1.js
first
third
second
PS C:\Users\Yash Jain\koss>
```

Code explanation

Synchronous -

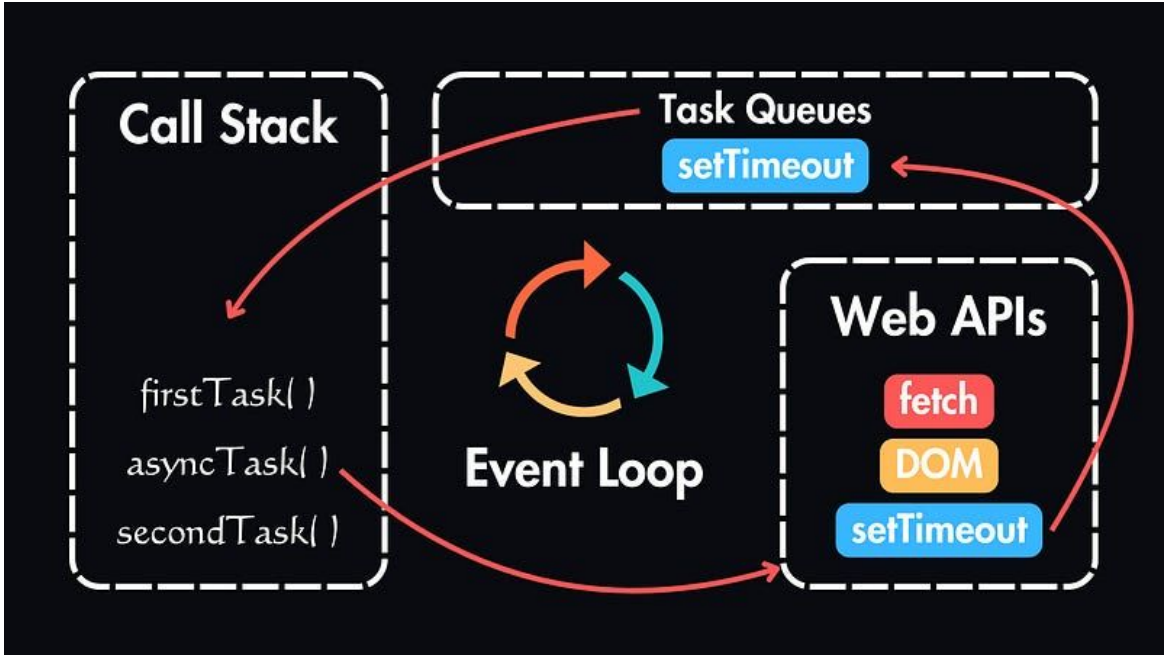
Every instruction runs once after the previous instruction gets executed. Due to this synchronous nature of javascript, we get the output of console logs in the sequence we provided in the program.

Asynchronous -

For the code , third gets printed before second, because of the asynchronous execution of the code. Here setTimeout() function waits for 2 seconds, and in the meantime, the next instruction gets executed without waiting for the previous one to complete the execution.

Event loop

The Event Loop is a fundamental mechanism that allows JavaScript—which is single-threaded—to perform non-blocking, asynchronous operations. It coordinates how code is executed, how events are collected and processed, and how queued sub-tasks are run.



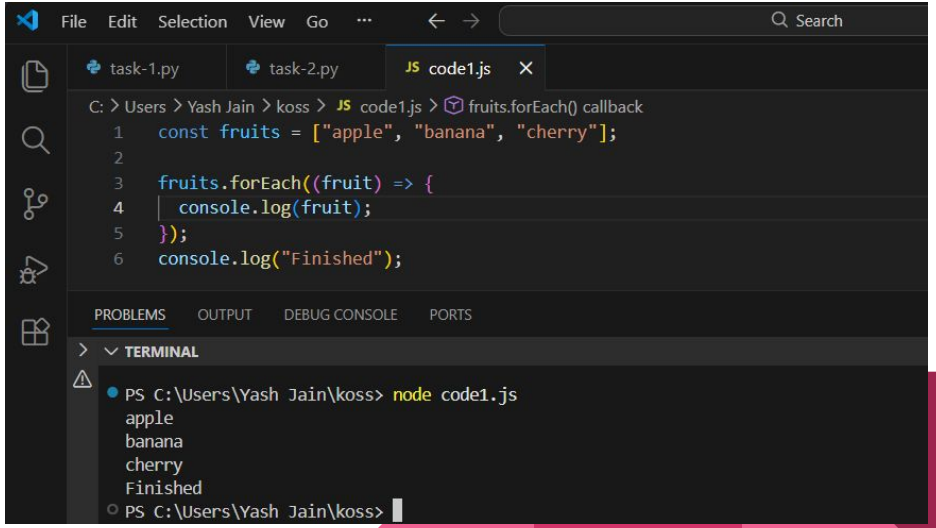
Callback functions

Def: A callback function is a function passed into another function as an argument, which is then invoked (called back) inside the outer function to complete a specific routine or action.

- Synchronous callbacks : These are executed immediately within the outer function, blocking further operations until completion. Array methods like map(), filter(), and forEach() use synchronous callbacks.
- Asynchronous callbacks : Asynchronous callbacks are executed at a later time, allowing the main program to continue running without waiting.

  This is essential for preventing the application from freezing during long-running tasks like network requests

  Callback functions are used to handle derefferd tasks.



Callback function (Cont.)

Uses of asynchronous callbacks :

- Event Handling : User interactions, such as button clicks or key presses, can be handled by providing a callback function to an event listener

```javascript
document.getElementById("myButton").addEventListener("click", function () {
    console.log("Button clicked!");
});
```

Here the anonymous function is a callback that runs when the button is clicked.

C: > Users > Yash Jain > koss > JS code1.js > myFunction

```javascript
setTimeout(myFunction, 3000);

function myFunction() {
  console.log("Calback function is running.")
}
```

TERMINAL

```
PS C:\Users\Yash Jain\koss> node code1.js
Calback function is running.
```

Here myFunction is a callback function passed as an argument to setTimeout().

Callback hell / Pyramid of doom

Callback Hell happens when multiple dependent asynchronous callbacks are nested.

Problems with Callback hell:

- Deep Nesting: Each subsequent asynchronous operation is placed inside the callback of the previous one, leading to a "sideways pyramid" or "Christmas tree" shape.
- Tight Coupling: Each step is heavily dependent on the results of the previous step, making it hard to move or refactor individual parts.
- Complex Error Handling: Errors must be handled manually at every level of nesting, which often leads to repetitive and messy code.

C: > Users > Yash Jain > koss > code1.js > step1() callback > step2() callback

```javascript
function step1(callback) {
    setTimeout(() => {
        console.log("Step 1 completed");
        callback();
    }, 1000);
}

function step2(callback) {
    setTimeout(() => {
        console.log("Step 2 completed");
        callback();
    }, 1000);
}

function step3(callback) {
    setTimeout(() => {
        console.log("Step 3 completed");
        callback();
    }, 1000);
}

step1(() => {
    step2(() => {
        step3(() => {
            console.log("All steps completed");
        });
    });
});
```

```
PS C:\Users\Yash Jain\koss> node code1.js
Step 1 completed
Step 2 completed
Step 3 completed
All steps completed
```

As the number of callback functions increases, the nesting grows deeper, making the code difficult to manage

Solution !!!

Promises

A Promise is a proxy for a value not necessarily known when the promise is created.

Promise states :

- Pending: initial state, neither fulfilled nor rejected.
- Fulfilled: operation completed successfully (with a value).
- Rejected: operation failed (with a reason/error)

When the async work finishes, the promise settles (moves from pending to either fulfilled or rejected). Handlers attached via .then() or .catch() will be called when that happens.

Promise constructor :

It takes function with two parameters :

- resolve(value): Sets the promise state to fulfilled and provides the result.
- reject(reason): Sets the promise state to rejected and provides the error reason.

```
const p = new Promise((resolve, reject) => {
  // Asynchronous work here...
  if (/* success */) resolve(value);
  else reject(error);
});
```

Promises cont.

Consuming Promises :

- .then(onFulfilled, onRejected): Attaches handlers for fulfillment and/or rejection. It returns a new promise, enabling chaining.
- .catch(onRejected): A shorthand for handling only rejections. It is typically placed at the end of a chain to catch any error that occurs.
- .finally(onFinally): Executes a callback regardless of whether the promise was fulfilled or rejected,

C: > Users > Yash Jain > koss > JS code1.js > ...

```javascript
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    const randomNumber = Math.random();
    if (randomNumber < 0.5) {
      resolve("Data has been successfully retrieved.");
    } else {
      reject("An error occurred while fetching data.");
    }
  }, 1000);
});

myPromise
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error(error);
  });
```

Promises cont.

Chaining :

- A key power of Promises is chaining multiple async operations in sequence. The .then() method returns a new promise each time .
- The then() method takes up to two arguments; the first argument is a callback function for the fulfilled case of the promise, and the second argument is a callback function for the rejected case. The catch() and finally() methods call then() internally and make error handling less verbose

Error Handling :

Promises simplify error handling. Instead of checking for errors in each callback, .catch() can handle the errors centrally. Once promise is rejected, all subsequent .then() blocks are skipped, and control moves directly to .catch().The .catch() handles the error, logs the message, and prevents the application from crashing.

C: > Users > Yash Jain > koss > JS code1.js > ...

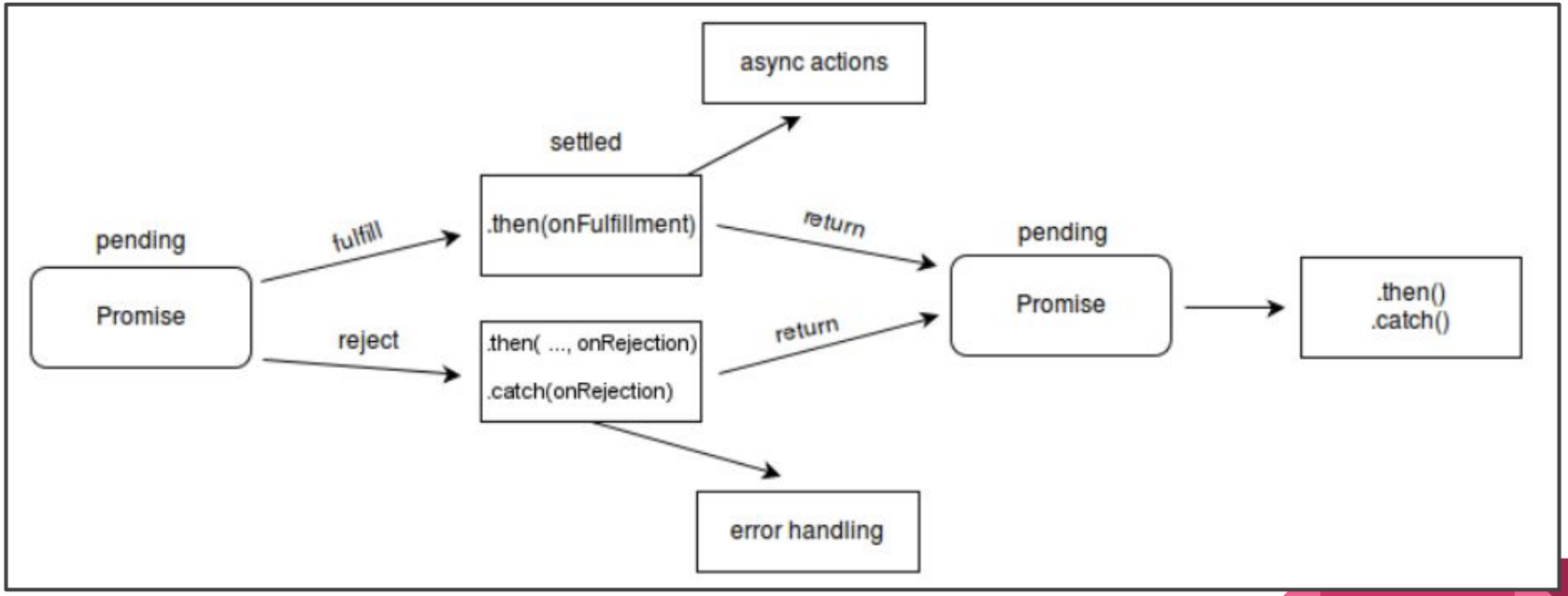
```js
new Promise((resolve) => {
  setTimeout(() => resolve(10), 1000); // Initial value: 10
})
  .then((result) => {
    console.log(result); // 10
    return result * 2;   // Pass 20 to the next .then()
  })
  .then((result) => {
    console.log(result); // 20
    return result + 5;   // Pass 25 to the next .then()
  })
  .then((result) => {
    console.log(result); // 25
  })
  .catch((error)=>{
    console.log("Error",error);
  });
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS

TERMINAL

```
PS C:\Users\Yash Jain\koss> node code1.js
10
20
25
PS C:\Users\Yash Jain\koss>
```

Promises cont.



Promises cont.

Benefits of promises over callbacks :

- **Avoid Callback Hell:** Promises organize asynchronous code more neatly than nested callbacks.
- **Centralised Error Handling:** Errors can be caught in one place using .catch() Method.
- **Chaining:** Perform tasks sequentially with .then() Method.

Callback hell:

```
getData(function(a) {
processData(a, function(b) {
saveData(b, function(c) {
console.log(c);
}, (err) => console.error(err));
}, (err) => console.error(err));
}, (err) => console.error(err));
```

Promises :

```
getData()
.then(processData)
.then(saveData)
.then(console.log);
.catch(err => console.error(err));
```

Async / await

- async / await is syntactic sugar over promises that makes asynchronous code read like synchronous code.
- An async function always returns a promise. The await operator pauses the async function until the awaited promise settles.
- Makes error handling simpler using try..catch
- Benefits -
  - Improved readability
  - Simplified error handling
  - Better debugging

Syntax:

```
async function functionName() {
 try {
   const result = await someAsyncFunction();
   console.log(result);
 } catch (error) {
   console.error("Error:", error.message);
 }
}
```

Async await cont.

- async function fetchData(): Declares an asynchronous function that always returns a Promise.
- await fetch(...): Sends an HTTP request and waits for the response without blocking the main thread.
- await response.json(): Converts the response body into JavaScript data (JSON format).
- try...catch: Handles errors such as network failure or invalid responses.

C: > Users > Yash Jain > koss > JS code1.js > fetchData

```javascript
async function fetchData() {
  try {
    let response = await fetch('https://api.example.com/data');
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error fetching data:', error);
  }
}
```

Async await cont.

Important behaviors: -

- await can only be used inside async functions .
- The await operator pauses the async function without blocking the main thread. Other code (including other async functions) can run while awaiting.
- If an awaited promise is rejected, await throws the rejection reason (as an exception), which can be catched.

Callback vs Promises vs Async/Await

	Callback	Promises	Async/Await
Definition	A function passed as an argument to another function.	An object representing the eventual completion/failure of a task.	Syntactic sugar built on top of Promises for a cleaner look.
Readability	Poor; often leads to "Callback Hell" (nested code).	Better; uses .then() chaining to keep code linear.	Best; looks and behaves like synchronous code.
Error handling	Manual checks (e.g., error-first arguments).	Uses .catch() to centralize error management.	Uses standard try...catch blocks.
Execution	Starts the task and provides a "callback" for when it finishes.	Returns a "placeholder" object immediately.	Pauses execution at await until the promise settles